IPD440 - APRENDIZAJE DE MAQUINAS

Clase 6: Decision Trees

Prof. Nicolás Torres
nicolas.torresr@usm.cl
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

Aprendizaje no paramétrico

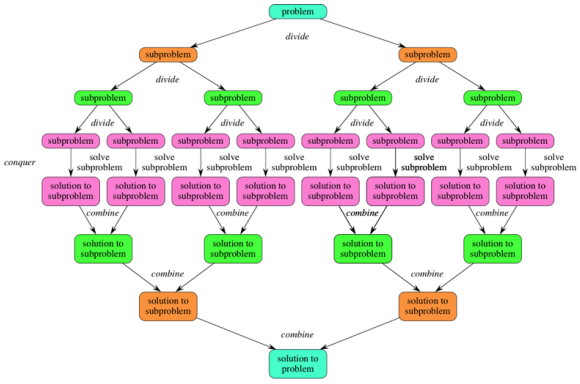
En la estimación **paramétrica**, definimos un modelo sobre todo el espacio de entrada y aprendemos sus parámetros de todos los datos de entrenamiento. Luego usamos el mismo modelo y el mismo conjunto de parámetros para cualquier dato de testing.

En la estimación **no paramétrica**, dividimos el espacio de entrada en regiones locales, definidas por una medida de distancia como la norma euclidiana, y para cada entrada, se usa el modelo local correspondiente calculado a partir de los datos de entrenamiento en esa región.

Ejemplos modelos no paramétricos:

- K-NN (K-nearest neighbors).
- SVM (Máquinas de vectores de soporte).
- Árboles de decisión.

"Divide y vencerás"



Árboles de Decisión

Los árboles de decisión (Decision Trees) son un método jerárquico de aprendizaje supervisado **no paramétrico** que se pueden utilizar tanto para clasificación como para regresión. El objetivo es crear un modelo que prediga el valor de una variable objetivo mediante el aprendizaje de reglas de decisión simples deducidas de las características de los datos. Se basan en el principio "Divide y vencerás":

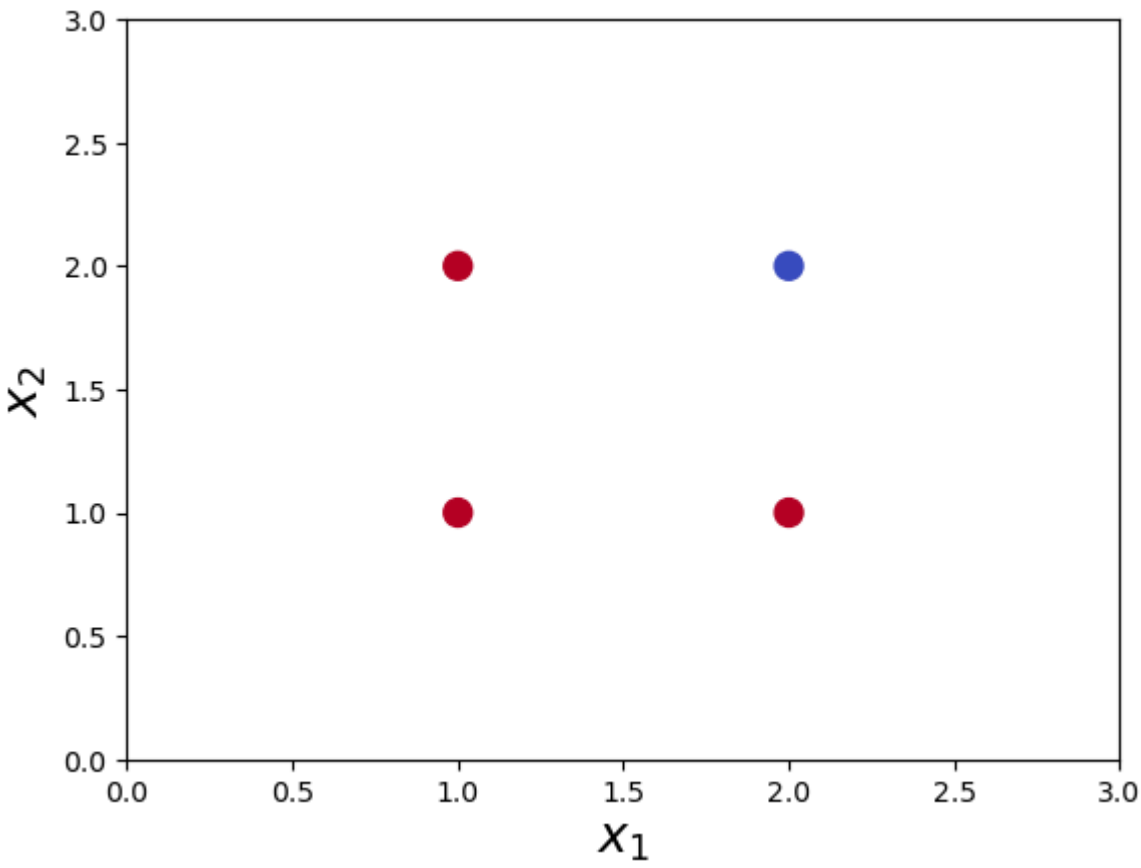
- Secuencia de divisiones recursivas en un número mínimo de pasos.
- Nodos de decisión interna y hojas terminales.
- Cada nodo hoja tiene una salida $\hat{y}$, que en el caso de la clasificación es la etiqueta de la clase y en la regresión es un valor numérico.

In [1]:
```
import pandas as pd
from sklearn import tree
import numpy as np
import matplotlib.pyplot as plt
```

Ejemplo

In [2]:
```
X, y = np.array([ [1,1],[1,2],[2,1],[2,2] ]), np.array([0,0,0,1])
```

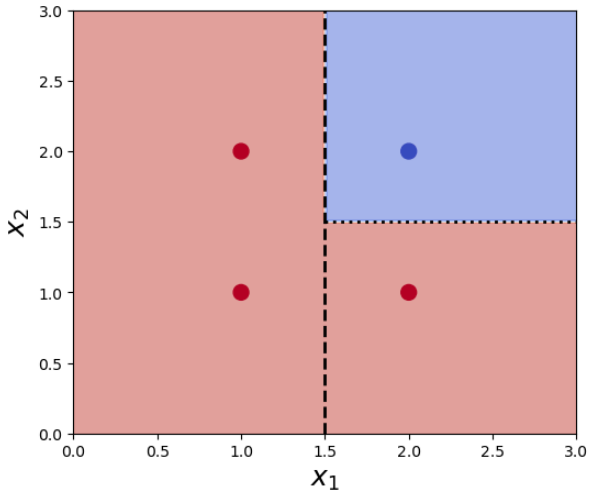
In [3]:
```
plt.scatter(X[:, 0], X[:, 1], s=100, c=y, cmap=plt.cm.coolwarm_r)
plt.xlabel("$x_1$", fontsize=18)
plt.ylabel("$x_2$", fontsize=18)
plt.xlim(0,3)
plt.ylim(0,3)
plt.show()
```

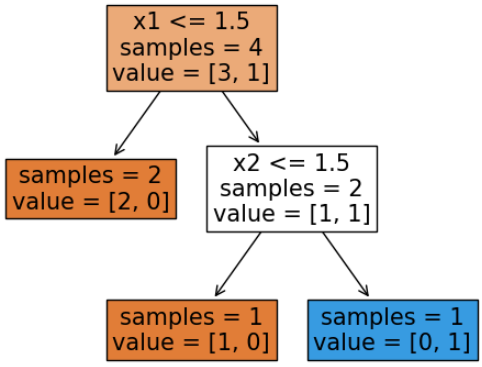


In [4]:
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.inspection import DecisionBoundaryDisplay
```

In [5]:
```
def plot_decision_tree(X, y):
    fig = plt.figure(figsize=(13, 5))
    ax1 = fig.add_subplot(121)
    ax2 = fig.add_subplot(122)
    tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42).fit(X, y)
    disp = DecisionBoundaryDisplay.from_estimator(tree_clf, X, cmap=plt.cm.coolwarm
                                                  response_method="predict",alpha=0
    disp.ax_.scatter(X[:, 0], X[:, 1], s=100, c=y, cmap=plt.cm.coolwarm_r)
    ax1.plot([1.5, 1.5], [0, 3], "k--", linewidth=2)
    ax1.plot([1.5, 3], [1.5, 1.5], "k:", linewidth=2)
    ax1.set_xlabel("$x_1$", fontsize=18)
    ax1.set_ylabel("$x_2$", fontsize=18)
    ax1.set_xlim(0,3)
    ax1.set_ylim(0,3)
    tree.plot_tree(tree_clf, feature_names=["x1","x2"], filled = True, impurity=Fal
    plt.show()
```

In [6]:
```
plot_decision_tree(X,y)
```



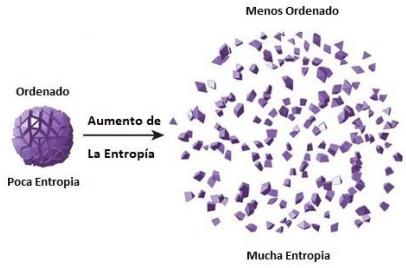


Árboles de Clasificación

En el caso de un árbol de decisión para la clasificación, llamado árbol de clasificación, la división se realiza tal que minimiza una función de impureza. Una división es pura si después de la división, para todas las ramas, todas las instancias que eligen una rama pertenecen a la misma clase. En este caso, no necesitamos dividir más y podemos agregar un nodo hoja etiquetado con la clase respectiva. Una posible función para medir la impureza es la entropía o el coeficiente de Gini.

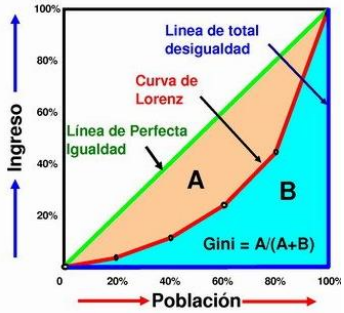
Entropía

La entropía es un término que proviene de la física y significa una medida de desorden.



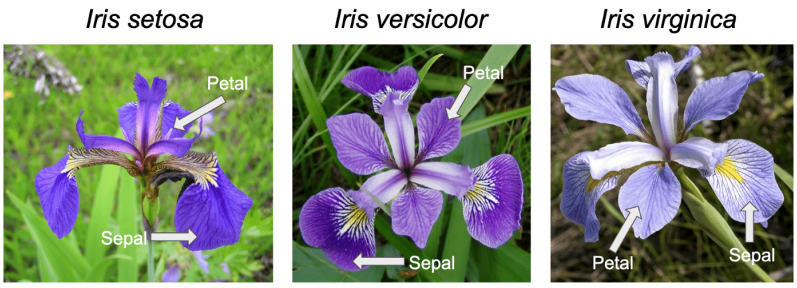
En el contexto de los árboles de decisión, la entropía es una medida de desorden o impureza en un nodo. Los nodos hoja que tienen todas las instancias pertenecientes a la misma clase, tendrán una entropía de 0.

Coeficiente de Gini



La otra forma de dividir un árbol de decisión es a través del Coeficiente de Gini (default en `sklearn`) que mide desigualdad, donde 0 indica que todos los elementos del nodo presentan la misma igualdad y 1 corresponde a la desigualdad perfecta.

Iris dataset



In [7]:
```
from sklearn.datasets import load_iris
```

In [8]:
```
iris = load_iris()
```

In [9]:
```
df_iris = pd.DataFrame(data=iris.data, columns=np.array(iris.feature_names))
```

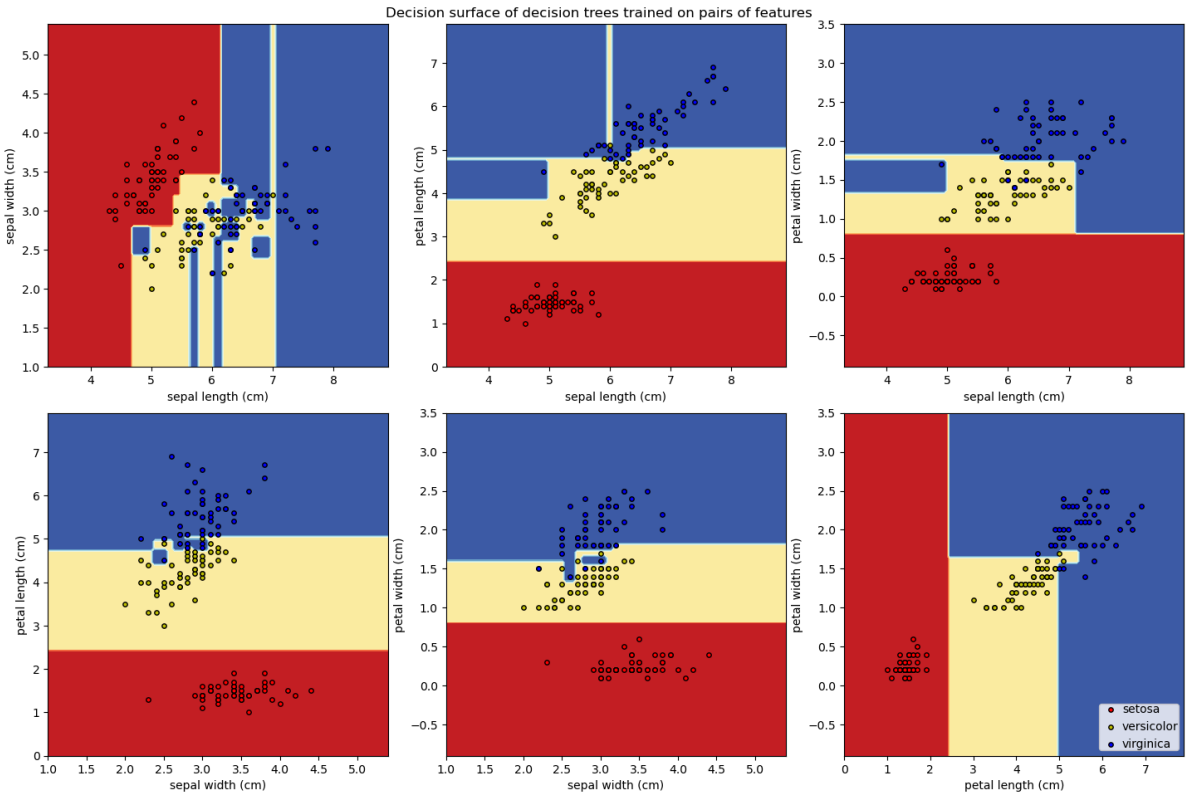
In [10]:
```
df_iris.tail()
```

Out[10]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

In [11]:
```
def plot_decision_surface(dataset, n_classes = 3, plot_colors = "ryb"):
    plt.figure(figsize=(15, 10))
    for pairidx, pair in enumerate([[0, 1], [0, 2], [0, 3], [1, 2], [1, 3], [2, 3]]
        X = dataset.data[:, pair]
        y = dataset.target
        clf = DecisionTreeClassifier().fit(X, y)
        ax = plt.subplot(2, 3, pairidx + 1)
        plt.tight_layout(h_pad=0.5, w_pad=0.5, pad=2.5)
        DecisionBoundaryDisplay.from_estimator(clf, X,
            cmap=plt.cm.RdYlBu, response_method="predict", ax=ax,
            xlabel=dataset.feature_names[pair[0]], ylabel=dataset.feature_names[pai
        for i, color in zip(range(n_classes), plot_colors):
            idx = np.where(y == i)
            plt.scatter(X[idx, 0], X[idx, 1], c=color, label=dataset.target_names[i
    plt.suptitle("Decision surface of decision trees trained on pairs of features")
    plt.legend(loc="lower right", borderpad=0, handletextpad=0)
    _ = plt.axis("tight")
    plt.show()
```

In [12]:
```
plot_decision_surface(iris)
```

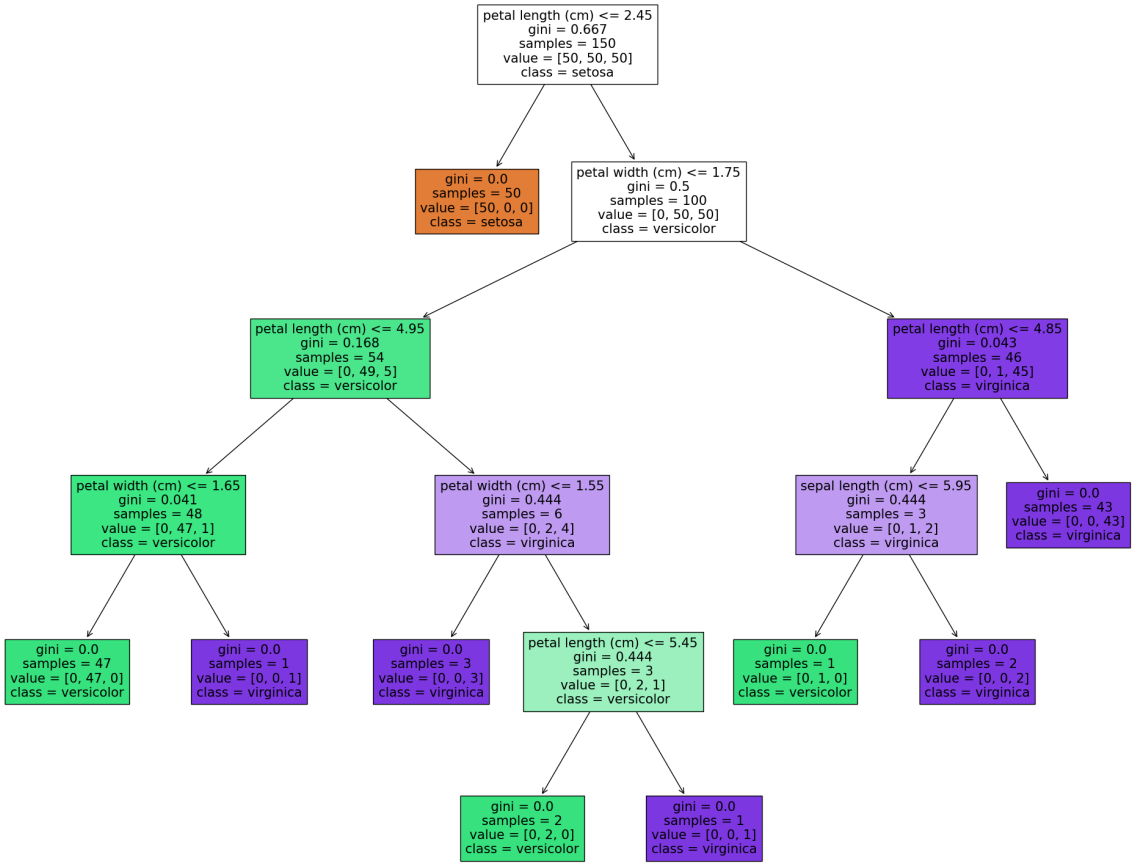


In [13]:
```
X_iris, y_iris = iris.data, iris.target
```

In [14]:
```
decision_tree_iris = DecisionTreeClassifier()
```

```
decision_tree_iris = decision_tree_iris.fit(X_iris, y_iris)
```

In [15]:
```
plt.figure(figsize=(25, 20))
tree.plot_tree(decision_tree_iris, feature_names = list(df_iris.columns), class_nam
plt.show()
```



Reglas de decisión

Un árbol de decisión hace su propia extracción de características. Podemos decir que las características más cercanas a la raíz son más importantes a nivel global.

In [16]:
```
rules_iris = tree.export_text(decision_tree_iris, feature_names=iris['feature_names
```

In [17]:
```
print(rules_iris)
```

```
|--- petal length (cm) <= 2.45
|   |--- class: setosa
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.75
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- petal width (cm) <= 1.65
|   |   |   |   |--- class: versicolor
|   |   |   |--- petal width (cm) >  1.65
|   |   |   |   |--- class: virginica
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- petal width (cm) <= 1.55
|   |   |   |   |--- class: virginica
|   |   |   |--- petal width (cm) >  1.55
|   |   |   |   |--- petal length (cm) <= 5.45
|   |   |   |   |   |--- class: versicolor
|   |   |   |   |--- petal length (cm) >  5.45
|   |   |   |   |   |--- class: virginica
|   |--- petal width (cm) >  1.75
|   |   |--- petal length (cm) <= 4.85
|   |   |   |--- sepal length (cm) <= 5.95
|   |   |   |   |--- class: versicolor
|   |   |   |--- sepal length (cm) >  5.95
|   |   |   |   |--- class: virginica
|   |   |--- petal length (cm) >  4.85
|   |   |   |--- class: virginica
```

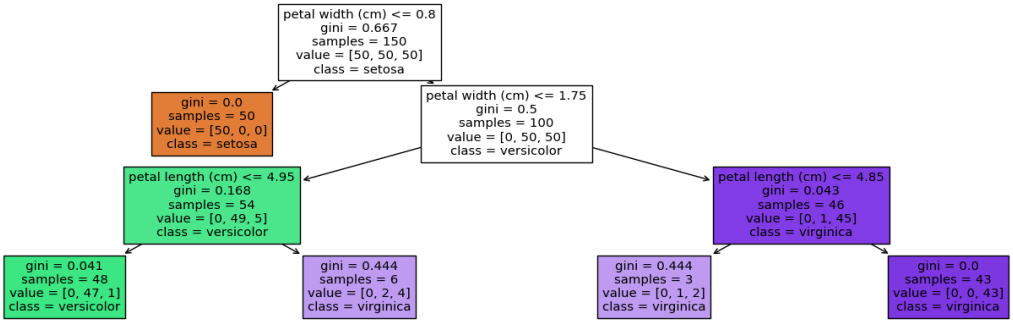
Poda

Con frecuencia, un nodo no se divide más si el número de instancias de entrenamiento que llegan a un nodo es menor que un determinado porcentaje del conjunto de entrenamiento. Usualmente se limita el crecimiento del árbol a través del parámetro `max_depth` para no caer en sobreajuste.

In [18]:
```
prunning_tree = DecisionTreeClassifier(random_state=0, max_depth=3).fit(iris.data,
prunning_rules_iris = tree.export_text(prunning_tree, feature_names=iris['feature_n
                                       class_names = list(iris.target_names))
print(prunning_rules_iris)
```

```
|--- petal width (cm) <= 0.80
|   |--- class: setosa
|--- petal width (cm) >  0.80
|   |--- petal width (cm) <= 1.75
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: versicolor
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- class: virginica
|   |--- petal width (cm) >  1.75
|   |   |--- petal length (cm) <= 4.85
|   |   |   |--- class: virginica
|   |   |--- petal length (cm) >  4.85
|   |   |   |--- class: virginica
```

In [19]:
```
plt.figure(figsize=(18, 5))
tree.plot_tree(prunning_tree, feature_names = list(df_iris.columns), class_names =
plt.show()
```



Titanic dataset

In [20]:
```
df_titanic = pd.read_csv('titanic.csv')
```

In [21]:
```
df_titanic
```

Out[21]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cal
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	N
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	N
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	N
...	...	...	...	...	...	...	...	...	...	...	
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	N
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	E
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	N
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C1
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	N

891 rows × 12 columns

In [22]:
```
data_titanic = df_titanic.loc[:,('Survived','Pclass','Sex','Age','SibSp','Parch','F
```

In [23]:
```
data_titanic.dropna(inplace=True)
```

In [24]:
```
from sklearn.preprocessing import LabelEncoder
```

In [25]:
```
le = LabelEncoder()
data_titanic.Sex = le.fit_transform(data_titanic.Sex)
```

In [26]:
```
data_titanic
```

Out[26]:

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare
0	0	3	1	22.0	1	0	7.2500
1	1	1	0	38.0	1	0	71.2833
2	1	3	0	26.0	0	0	7.9250
3	1	1	0	35.0	1	0	53.1000
4	0	3	1	35.0	0	0	8.0500
...	...	...	...	...	...	...	...
885	0	3	0	39.0	0	5	29.1250
886	0	2	1	27.0	0	0	13.0000
887	1	1	0	19.0	0	0	30.0000
889	1	1	1	26.0	0	0	30.0000
890	0	3	1	32.0	0	0	7.7500

714 rows × 7 columns

In [27]:
```
X_titanic = data_titanic.iloc[:,1:]
y_titanic = data_titanic.iloc[:,0]
```

In [28]:
```
from sklearn.model_selection import train_test_split
```

In [29]:
```
X_train_titanic, X_test_titanic, y_train_titanic, y_test_titanic = train_test_split
```

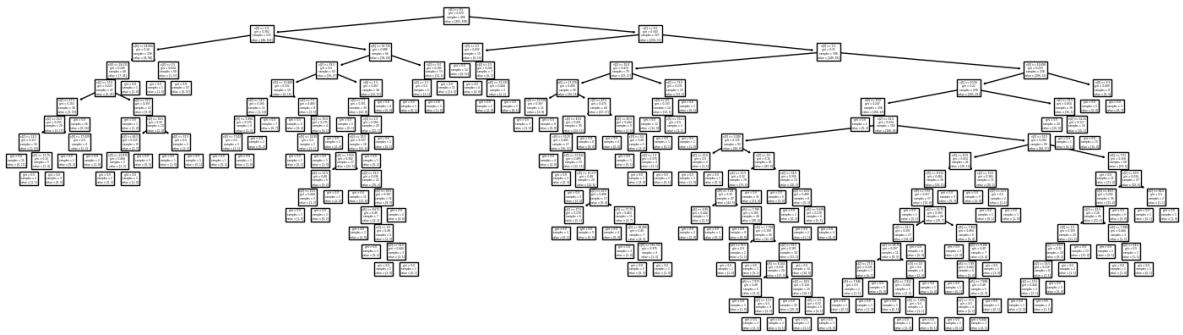
In [30]:
```
dtree_titanic = DecisionTreeClassifier()
dtree_titanic.fit(X_train_titanic, y_train_titanic)
y_pred_titanic = dtree_titanic.predict(X_test_titanic)
```

In [31]:
```
from sklearn.metrics import accuracy_score
```

In [32]:
```
accuracy_score(y_test_titanic, y_pred_titanic)
```

Out[32]:
```
0.8
```

In [33]:
```
plt.figure(figsize=(18, 5))
tree.plot_tree(dtree_titanic)
plt.show()
```



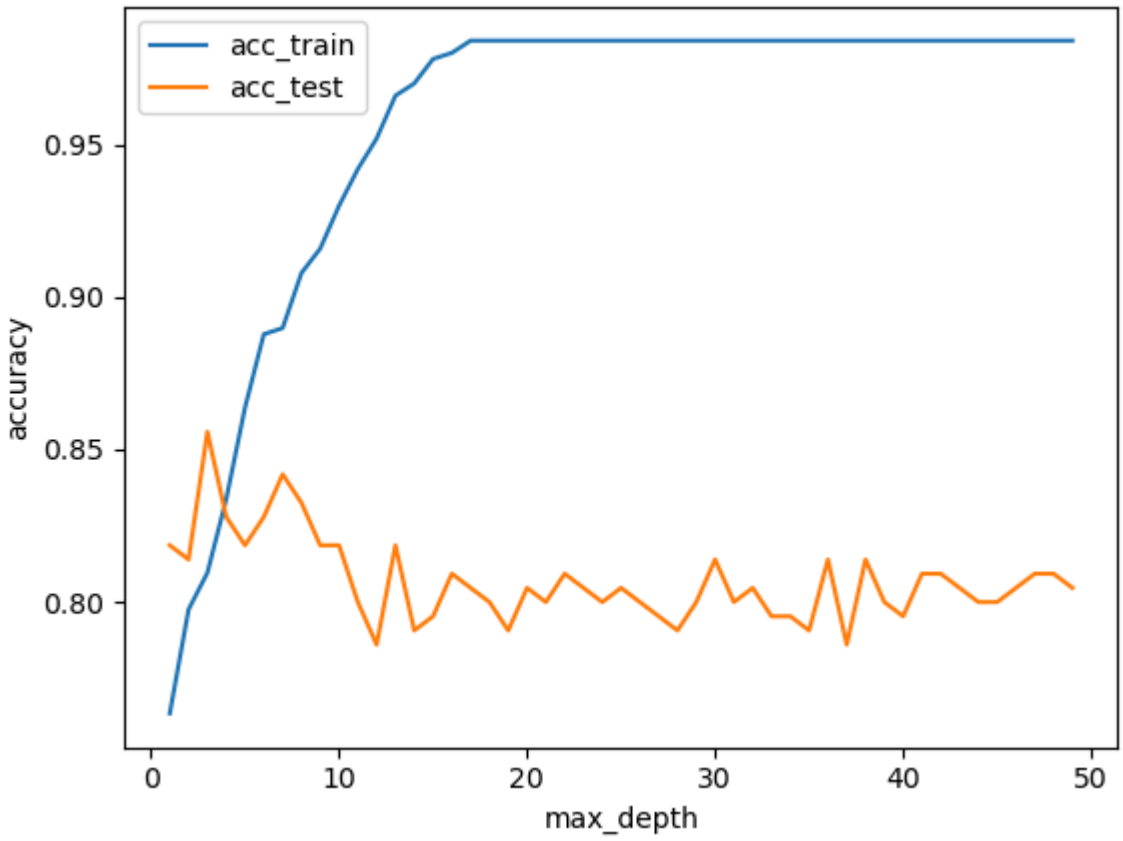
In [34]:
```
max_depth = []
acc_train = []
```

```
acc_test = []

for i in range(1,50):
    dtree = DecisionTreeClassifier(max_depth=i)
    dtree.fit(X_train_titanic, y_train_titanic)
    y_pred_train = dtree.predict(X_train_titanic)
    acc_train.append(accuracy_score(y_train_titanic, y_pred_train))
    y_pred_test = dtree.predict(X_test_titanic)
    acc_test.append(accuracy_score(y_test_titanic, y_pred_test))
    max_depth.append(i)

d = pd.DataFrame({'acc_train':pd.Series(acc_train),
                  'acc_test':pd.Series(acc_test),
                  'max_depth':pd.Series(max_depth)})
```

In [35]:
```
plt.plot('max_depth','acc_train', data=d, label='acc_train')
plt.plot('max_depth','acc_test', data=d, label='acc_test')
plt.xlabel('max_depth')
plt.ylabel('accuracy')
plt.legend()
plt.show()
```



Criterio de división

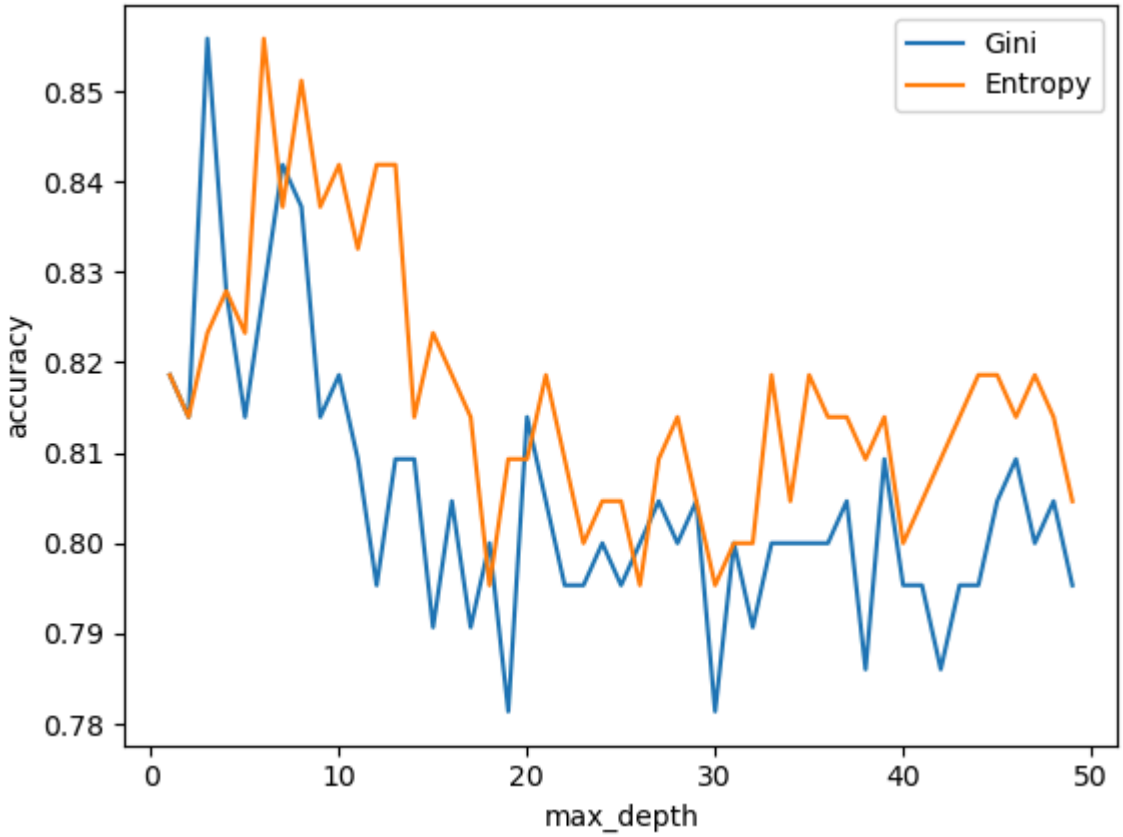
In [36]:
```
max_depth = []
acc_gini = []
acc_entropy = []

for i in range(1,50):
    dtree = DecisionTreeClassifier(criterion='gini', max_depth=i)
    dtree.fit(X_train_titanic, y_train_titanic)
    pred = dtree.predict(X_test_titanic)
    acc_gini.append(accuracy_score(y_test_titanic, pred))
```

```
    dtree = DecisionTreeClassifier(criterion='entropy', max_depth=i)
    dtree.fit(X_train_titanic, y_train_titanic)
    pred = dtree.predict(X_test_titanic)
    acc_entropy.append(accuracy_score(y_test_titanic, pred))
    max_depth.append(i)

d_titanic = pd.DataFrame({'acc_gini':pd.Series(acc_gini),
                          'acc_entropy':pd.Series(acc_entropy),
                          'max_depth':pd.Series(max_depth)})
```

In [37]:
```
plt.plot('max_depth','acc_gini', data=d_titanic, label='Gini')
plt.plot('max_depth','acc_entropy', data=d_titanic, label='Entropy')
plt.xlabel('max_depth')
plt.ylabel('accuracy')
plt.legend()
plt.show()
```



GridSearchCV

Búsqueda exhaustiva sobre valores de parámetros específicos para un estimador. Permite hacer pruebas con diferentes configuraciones para encontrar los parámetros óptimos del modelo.

In [38]:
```
from sklearn.model_selection import GridSearchCV
```

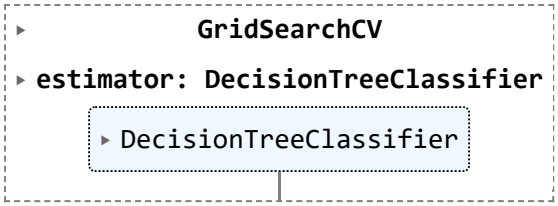
In [39]:
```
param_grid = {'max_depth': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 20, 30, 50, 75, 100]
```

In [40]:
```
dtree = DecisionTreeClassifier()
```

In [41]:
```
grid_search = GridSearchCV(estimator = dtree, param_grid = param_grid, cv = 5)
```

```
grid_search.fit(X_train_titanic, y_train_titanic)
```

Out[41]:



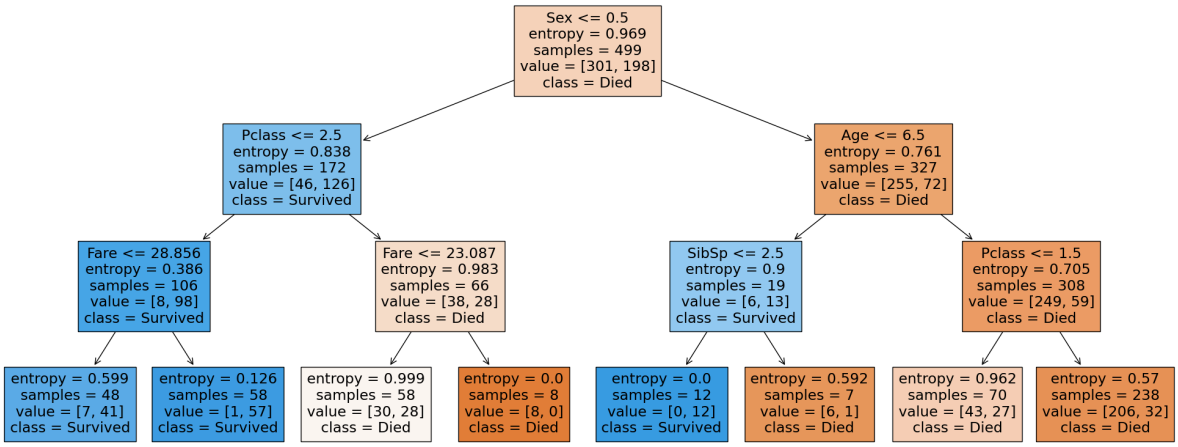
In [42]:
```
grid_search.best_params_
```

Out[42]:
```
{'max_depth': 3}
```

In [43]:
```
dtree_titanic = DecisionTreeClassifier(criterion='entropy', max_depth=3)
dtree_titanic.fit(X_train_titanic, y_train_titanic)
y_pred_titanic = dtree_titanic.predict(X_test_titanic)
accuracy_score(y_test_titanic, y_pred_titanic)
```

Out[43]:
```
0.8232558139534883
```

In [44]:
```
plt.figure(figsize=(25, 10))
tree.plot_tree(dtree_titanic, feature_names = list(data_titanic.iloc[:,1:].columns)
               class_names = ['Died','Survived'], filled = True)
plt.show()
```



In [45]:
```
rules_titanic = tree.export_text(dtree_titanic, feature_names=list(data_titanic.ilc
                                 class_names = ['Died','Survived'])
print(rules_titanic)
```

```
|--- Sex <= 0.50
|   |--- Pclass <= 2.50
|   |   |--- Fare <= 28.86
|   |   |   |--- class: Survived
|   |   |--- Fare >  28.86
|   |   |   |--- class: Survived
|   |--- Pclass >  2.50
|   |   |--- Fare <= 23.09
|   |   |   |--- class: Died
|   |   |--- Fare >  23.09
|   |   |   |--- class: Died
|--- Sex >  0.50
|   |--- Age <= 6.50
|   |   |--- SibSp <= 2.50
|   |   |   |--- class: Survived
|   |   |--- SibSp >  2.50
|   |   |   |--- class: Died
|   |--- Age >  6.50
|   |   |--- Pclass <= 1.50
|   |   |   |--- class: Died
|   |   |--- Pclass >  1.50
|   |   |   |--- class: Died
```

Árboles de Regresión

Los árboles de decisión también se pueden aplicar a problemas de regresión, utilizando la clase `DecisionTreeRegressor`.

Al igual que en la configuración de clasificación, el método de ajuste tomará como argumentos las matrices `X` e `y`, solo que en este caso se espera que `y` tenga valores de coma flotante en lugar de valores enteros.

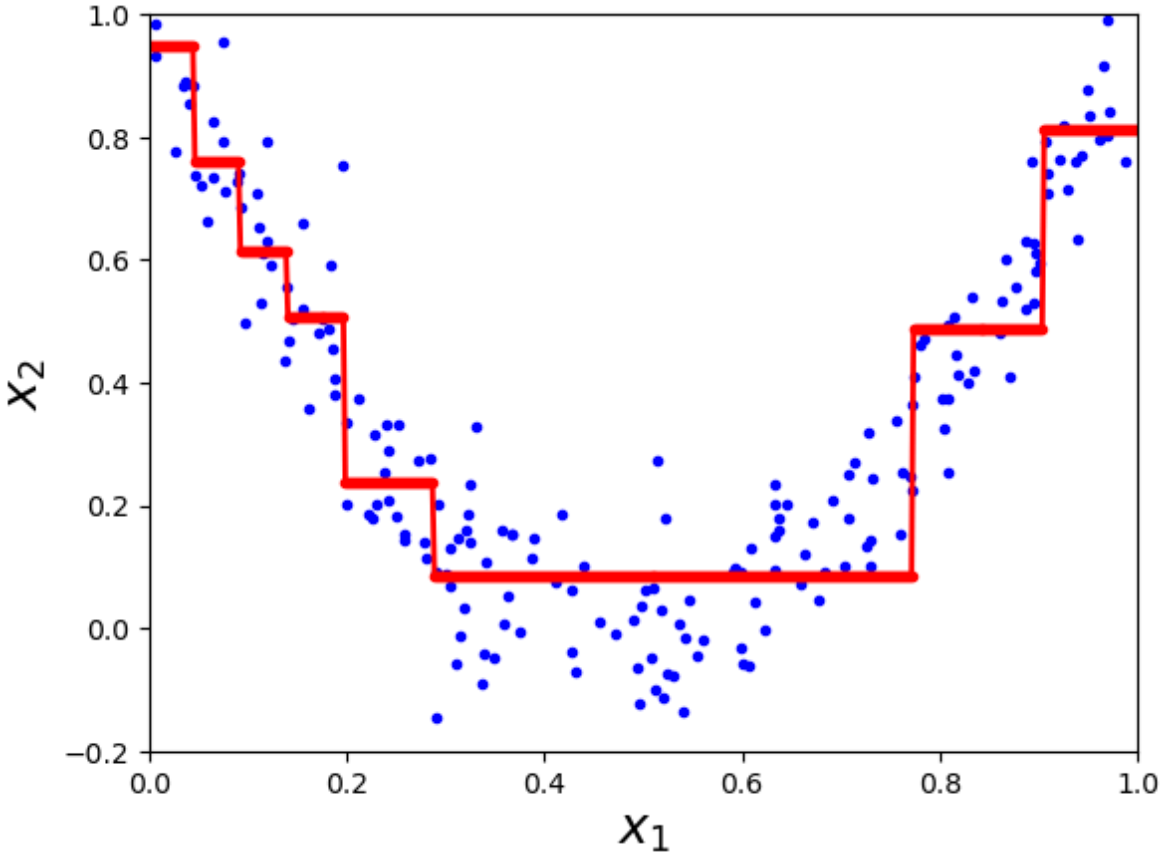
Un árbol de regresión se construye casi de la misma manera que un árbol de clasificación, excepto que la medida de impureza que es apropiada para la clasificación se reemplaza por una medida apropiada para la regresión como Mean Square Error (MSE).

In [46]:
```
np.random.seed(42)
m = 200
X = np.random.rand(m, 1)
y = 4 * (X - 0.5) ** 2
y = y + np.random.randn(m, 1) / 10
```

In [47]:
```
from sklearn.tree import DecisionTreeRegressor
```

In [48]:
```
def plot_regression_predictions(X, y, max_depth=None, min_samples_leaf=10, axes=[0,
    x1 = np.linspace(axes[0], axes[1], 500).reshape(-1, 1)
    tree_reg = DecisionTreeRegressor(max_depth=max_depth, min_samples_leaf=min_samp
    y_pred = tree_reg.predict(x1)
    plt.axis(axes)
    plt.plot(X, y, "b.")
    plt.xlabel("$x_1$", fontsize=18)
    plt.ylabel("$x_2$", fontsize=18)
    plt.plot(x1, y_pred, "r.-", linewidth=2)
```

In [49]:
```
plot_regression_predictions(X, y, max_depth=3)
```



In [50]:
```
tree_reg = DecisionTreeRegressor(max_depth=3, random_state=42).fit(X, y)
plt.figure(figsize=(25, 10))
tree.plot_tree(tree_reg, feature_names=["x1","x2"], filled = True)
plt.show()
```

